

FORMATION EMBARQUÉE

TP — tp1 arduino station meteo

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 1 — Station météo Arduino/ESP32 (≈ 12 h, en 4 séances)

Objectif pédagogique : construire un système embarqué complet et non bloquant — capteur, affichage, journalisation, réseau — en appliquant `millis()`, la FSM, I2C, SPI et MQTT. C'est le projet vitrine du début de portfolio.

 **Version détaillée** : chaque séance existe en fiche minutée avec code complet, erreurs fréquentes et travail à la maison — [Séance 1](#) · [Séance 2](#) · [Séance 3](#) · [Séance 4](#). Cette page reste la vue d'ensemble et la grille de notation.

Matériel / alternative simulateur

- Arduino Uno ou Nano (séances 1-3), ESP32 (séance 4)
- DHT22, écran OLED SSD1306 (I2C), potentiomètre, LED rouge, buzzer (option), module carte SD (option séance 3)
- **Sans matériel** : tout le TP (séance 4 comprise) se fait sur <https://wokwi.com> — crée un projet « Arduino Uno » puis « ESP32 ».

Bibliothèques : `DHT sensor library` (Adafruit), `Adafruit SSD1306`, `Adafruit GFX`, et séance 4 : `PubSubClient`.

Règle du TP : un seul `delay()` toléré dans tout le projet (celui du `setup()` éventuel). Chaque séance se termine par un commit Git.

Séance 1 (2 h) — Socle non bloquant

Étape 1.1 — Câblage

DHT22 : VCC → 5V, GND → GND, DATA → D2 (avec pull-up 10 kΩ si module nu). LED rouge : D8 → résistance 220 Ω → LED → GND. Potentiomètre : extrémités 5V/GND, curseur → A0.

Étape 1.2 — Squelette à tâches périodiques

Écris le squelette suivant et **vérifie au moniteur série que les deux périodes tiennent indépendamment** :

```

#include <DHT.h>
DHT dht(2, DHT22);

uint32_t t_mesure = 0, t_affiche = 0;
float temperature = NAN, humidite = NAN;

void setup() {
    Serial.begin(115200);
    dht.begin();
    pinMode(8, OUTPUT);
}

void loop() {
    uint32_t now = millis();

    if (now - t_mesure >= 2000) {          // tâche 1 : mesurer (2 s)
        t_mesure = now;
        float t = dht.readTemperature(), h = dht.readHumidity();
        if (!isnan(t)) temperature = t;
        if (!isnan(h)) humidite = h;
    }

    if (now - t_affiche >= 1000) {        // tâche 2 : afficher (1 s)
        t_affiche = now;
        Serial.print("T="); Serial.print(temperature);
        Serial.print("C H="); Serial.print(humidite); Serial.println("%");
    }
}

```

✓ **Point de contrôle 1** : débranche la broche DATA du DHT en cours de fonctionnement. Le programme doit continuer d'afficher (les dernières valeurs valides), pas se figer ni afficher `nan` en boucle. Si ce n'est pas le cas, revois la gestion de `isnan`.

Étape 1.3 — Seuil et hystérésis

Ajoute : consigne = potentiomètre (10-40 °C), LED rouge allumée si `temperature > consigne + 0,5` et éteinte si `< consigne - 0,5` (reprends le TD 03 exercice 3). ✓ **Point de contrôle 2** : en soufflant sur le capteur, la LED ne « bat » pas autour du seuil.

Séance 2 (3 h) — Affichage OLED et architecture

Étape 2.1 — OLED I2C

SDA → A4, SCL → A5 (Uno). Adresse 0x3C. Affiche T/H en gros, consigne en petit, un pictogramme d'alerte si dépassement. Rafraîchis l'écran **au plus 5 fois/seconde** (tâche périodique dédiée) — jamais à chaque tour.

Étape 2.2 — Refactorisation en modules

Impose-toi la structure de fichiers suivante (onglets de l'IDE ou fichiers `.h/.cpp`) :

```
station_meteo/  
├─ station_meteo.ino      // setup/loop : UNIQUEMENT l'orchestration  
├─ capteur.h / .cpp      // lecture DHT + validité + dernier relevé  
├─ affichage.h / .cpp    // tout l'OLED (personne d'autre n'inclut Adafruit_SSD1306)  
└─ alerte.h / .cpp       // hystérésis + LED (+ buzzer)
```

✓ **Point de contrôle 3** : le `.ino` fait moins de 60 lignes et ne contient aucun appel direct aux bibliothèques Adafruit/DHT. Critère d'architecture : « pour changer d'écran, je ne touche qu'à `affichage.cpp` ».

Séance 3 (3 h) — FSM d'interface et journalisation

Étape 3.1 — Bouton et machine d'états d'affichage

Ajoute un bouton (D3, `INPUT_PULLUP`, anti-rebond du TD 03). Chaque appui change de page : `PAGE_MESURES → PAGE_MINMAX → PAGE_CONSIGNE → ...`. Implémente en `enum class` + `switch` (FSM du module 01). La page MIN/MAX affiche les extrêmes depuis le démarrage, remis à zéro par appui long (> 2 s — à détecter avec `millis()`, évidemment).

Étape 3.2 — (option matériel) Carte SD

Module SD en SPI (CS=D10). Toutes les 60 s, ajoute une ligne CSV : `millis;temperature;humidite`. Gère l'absence/retrait de carte sans planter (l'écriture échoue → on continue, on réessaie plus tard, on le signale sur la page d'accueil).

✓ **Point de contrôle 4** : appui court = page suivante, appui long = RAZ min/max, et le tout reste fluide pendant les mesures et l'écriture SD.

Séance 4 (4 h) — Version connectée ESP32 + MQTT

Étape 4.1 — Portage

Ouvre un projet ESP32 (Wokwi : carte « ESP32 DevKit »). Adapte : DHT sur GPIO 4, OLED sur 21/22 (I2C par défaut), LED sur GPIO 2. Tes modules `capteur/affichage/alerte` doivent passer **sans modification** — c'est le test de la séance 2.

Étape 4.2 — Wi-Fi + MQTT

```
#include <WiFi.h>
#include <PubSubClient.h>

WiFiClient net;
PubSubClient mqtt(net);

void connecter() {
    WiFi.begin("Wokwi-GUEST", ""); // réseau simulé de Wokwi
    while (WiFi.status() != WL_CONNECTED) delay(250); // seul delay toléré (init)
    mqtt.setServer("broker.hivemq.com", 1883); // broker public de test
    mqtt.connect("station-TONPRENOM"); // id UNIQUE, sinon déconnexions
}

void publier(float t, float h) {
    char json[64];
    snprintf(json, sizeof json, "{\"t\":%.1f,\"h\":%.1f}", t, h);
    mqtt.publish("formation/TONPRENOM/meteo", json);
}
```

Publie toutes les 10 s (tâche périodique). Appelle `mqtt.loop()` à chaque tour de `loop()`. Gère la reconnexion : si `!mqtt.connected()`, retente **au plus une fois toutes les 5 s** (pas en rafale).

Étape 4.3 — Vérification de bout en bout

Sur ton PC : `mosquitto_sub -h broker.hivemq.com -t "formation/TONPRENOM/#" -v` (ou l'outil web [hivemq.com/demos/websocket-client](https://broker.hivemq.com/demos/websocket-client)).  **Point de contrôle 5** : les JSON arrivent, et coupent proprement quand tu « débranches » le Wi-Fi dans la simulation, puis reprennent seuls.

Grille d'auto-évaluation du TP (à remplir honnêtement)

Critère	Points
Aucun <code>delay()</code> hors init ; périodes respectées	/4
Panne capteur, absence SD, perte Wi-Fi : gérées sans blocage	/4
Architecture en modules, <code>.ino</code> d'orchestration seul	/4
FSM d'affichage propre (enum + switch, transitions nettes)	/3
MQTT fonctionnel avec reconnexion raisonnée	/3
Git : ≥ 4 commits datés, messages clairs, README avec photo/schéma	/2
Total	/20

Pour aller plus loin : OTA (mise à jour par Wi-Fi), sommeil profond de l'ESP32 entre deux mesures (autonomie sur batterie), Node-RED côté PC pour tracer les courbes — chacun de ces ajouts est un futur commit de portfolio.